Veridi Logistics: Delivery Performance Audit

The CEO suspects our delivery estimates are over-promising and wants to know whether late deliveries are a nationwide issue or concentrated in specific regions. I'm using the Olist e-commerce dataset to connect the logistics side (when packages actually arrived vs. when we said they would) with the customer side (review scores), and follow the problem to its root cause.

The flow: build one clean master table → measure how badly we miss our promises → break it down by state → check what it does to reviews → figure out who's actually responsible.

Setup & Data Loading

I'm pulling the data straight from Kaggle with `kagglehub` instead of reading local files, so anyone can rerun this notebook top to bottom without downloading anything first. I also parse the date columns at load time the whole analysis is date arithmetic, so I'd rather deal with types once, here, than fight them in every cell below.

```
import kagglehub
path = kagglehub.dataset_download("olistbr/brazilian-ecommerce")
print(path)
```

```
Using Colab cache for faster access to the 'brazilian-ecommerce' dataset.
/kaggle/input/brazilian-ecommerce
```

```
import pandas as pd
import os

date_cols = [
    "order_purchase_timestamp",
    "order_approved_at",
    "order_delivered_carrier_date",
```

```
    "order_delivered_customer_date",
    "order_estimated_delivery_date",
]

orders    = pd.read_csv(os.path.join(path, "olist_orders_dataset.csv"), parse_dates=date_cols)
reviews   = pd.read_csv(os.path.join(path, "olist_order_reviews_dataset.csv"),
                        parse_dates=["review_creation_date", "review_answer_timestamp"])
customers = pd.read_csv(os.path.join(path, "olist_customers_dataset.csv"))
products  = pd.read_csv(os.path.join(path, "olist_products_dataset.csv"))

print(f"orders:    {orders.shape}")
print(f"reviews:   {reviews.shape}")
print(f"customers: {customers.shape}")
print(f"products:  {products.shape}")
orders.head(5)
```

```
orders:    (99441, 8)
reviews:   (99224, 7)
customers: (99441, 5)
products:  (32951, 9)
```

	order_id	customer_id	order_status	order_purchase_timesta
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:56
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:41
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:38
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:28
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:18

Next steps: Generate code with orders | New interactive sheet

Story 1: Building the Master Table

I need location and review score in the same row, so: Reviews join Orders on `order_id`, Customers join Orders on `customer_id`.

The dangerous part is the reviews join. It turns out 551 orders have more than one review, so a naive join would duplicate those orders and quietly skew every percentage I calculate afterwards. My fix: keep only the most recent review per order. My reasoning is that the latest review reflects the customer's final verdict, often left after the package finally showed up.

I also use left joins rather than inner ones, because 768 orders have no review at all. Dropping them here would throw away perfectly good delivery data; they just won't participate in the sentiment analysis later.

The asserts at the end are my safety net: if the master table doesn't have exactly one row per order, I want the notebook to crash loudly, not fail silently.

```
dupes = reviews["order_id"].duplicated().sum()
print(f"Orders with more than one review: {dupes}")
```

```
Orders with more than one review: 551
```

```
reviews_dedup = (
    reviews.sort_values("review_creation_date")
           .drop_duplicates(subset="order_id", keep="last")
)

master = (
    orders
    .merge(reviews_dedup[["order_id", "review_score", "review_creation_date"]],
           on="order_id", how="left")
    .merge(customers[["customer_id", "customer_city", "customer_state"]],
           on="customer_id", how="left")
)

assert len(master) == len(orders), "Row count changed — a join duplicated rows!"
assert master["order_id"].is_unique, "order_id is no longer unique!"

print(f"Master dataset: {master.shape}")
```

```
print(f"Master dataset: {master.shape}")
print(f"Orders without a review: {master['review_score'].isna().sum()}")
master.head(30)
```

```
Master dataset: (99441, 12)
Orders without a review: 768
```

	order_id	customer_id	order_status	order_purchase_times
0	e481f51cbdc54678b7cc49136f2d6af7	9ef432eb6251297304e76186b10a928d	delivered	2017-10-02 10:
1	53cdb2fc8bc7dce0b6741e2150273451	b0830fb4747a6c6d20dea0b8c802d7ef	delivered	2018-07-24 20:
2	47770eb9100c2d0c44946d9cf07ec65d	41ce2a54c0b03bf3443c3d931a367089	delivered	2018-08-08 08:
3	949d5b44dbf5de918fe9c16f97b45f8a	f88197465ea7920adcdbec7375364d82	delivered	2017-11-18 19:
4	ad21c59c0840e6cb83a9ceb5573f8159	8ab97904e6daea8866dbdbc4fb7aad2c	delivered	2018-02-13 21:
5	a4591c265e18cb1dcee52889e2d8acc3	503740e9ca751ccdda7ba28e9ab8f608	delivered	2017-07-09 21:
6	136cce7faa42fdb2cefd53fdc79a6098	ed0271e0b7da060a393796590e7b737a	invoiced	2017-04-11 12:
7	6514b8ad8028c9f2cc2374ded245783f	9bdf08b4b3b52b5526ff42d37d47f222	delivered	2017-05-16 13:
8	76c6e866289321a7c93b82b54852dc33	f54a9f0e6b351c431402b8461ea51999	delivered	2017-01-23 18:
9	e69bfb5eb88e0ed6a785585b27e16dbf	31ad1d1b63eb9962463f764d4e6e0c9d	delivered	2017-07-29 11:
10	e6ce16cb79ec1d90b1da9085a6118aeb	494dded5b201313c64ed7f100595b95c	delivered	2017-05-16 19:
11	34513ce0c4fab462a55830c0989c7edb	7711cf624183d843aafe81855097bc37	delivered	2017-07-13 19
12	82566a660a982b15fb86e904c8d32918	d3e3b74c766bc6214e0c830b17ee2341	delivered	2018-06-07 10:
13	5ff96c15d0b717ac6ad1f3d77225a350	19402a48fe860416adf93348aba37740	delivered	2018-07-25 17:
14	432aaf21d85167c2c86ec9448c4e42cc	3df704f53d3f1d4818840b34ec672a9f	delivered	2018-03-01 14:
15	dcb36b511fcac050b97cd5c05de84dc3	3b6828a50ffe546942b7a473d70ac0fc	delivered	2018-06-07 19:
16	403b97836b0c04a622354cf531062e5f	738b086814c6fcc74b8cc583f8516ee3	delivered	2018-01-02 19:
17	116f0b09343b49556bbad5f35bee0cdf	3187789bec990987628d7a9beb4dd6ac	delivered	2017-12-26 23:

18	85ce859fd6dc634de8d2f1e290444043	059f7fc5719c7da6cbafe370971a8d70	delivered	2017-11-21 00
19	83018ec114eee8641c97e08f7b4e926f	7f8c8b9c2ae27bf3300f670c3d478be8	delivered	2017-10-26 15
20	203096f03d82e0dffbc41ebc2e2bcfb7	d2b091571da224a1b36412c18bc3bbfe	delivered	2017-09-18 14
21	f848643eec1d69395095eb3840d2051e	4fa1cd166fa598be6de80fa84eaade43	delivered	2018-03-15 08
22	2807d0e504d6d4894d41672727bc139f	72ae281627a6102d9b3718528b420f8a	delivered	2018-02-03 20
23	95266dbfb7e20354baba07964dac78d5	a166da34890074091a942054b36e4265	delivered	2018-01-08 07
24	f3e7c359154d965827355f39d6b1fdac	62b423aab58096ca514ba6aa06be2f98	delivered	2018-08-09 11
25	fbf9ac61453ac646ce8ad9783d7d0af6	3a874b4d4c4b6543206ff5d89287f0c3	delivered	2018-02-20 23
26	acce194856392f074dbf9dada14d8d82	7e20bf5ca92da68200643bda76c504c6	delivered	2018-06-04 00
27	dd78f560c270f1909639c11b925620ea	8b212b9525f9e74e85e37ed6df37693e	delivered	2018-03-12 01
28	91b2a010e1e45e6ba3d133fa997597be	cce89a605105b148387c52e286ac8335	delivered	2018-05-02 11
29	ecab90c9933c58908d3d6add7c6f5ae3	761df82feda9778854c6dafdaeb567e4	delivered	2018-02-25 13

Next steps: Generate code with `master` New interactive sheet

Story 2: How Badly Do We Miss Our Promises?

`Days_Difference` = estimated delivery date − actual delivery date. Positive means we beat the promise, negative means we broke it. From there I bucket every order: **On Time** (met the estimate), **Late** (1–5 days past it), and **Super Late** (more than 5 days past).

About 3% of orders were never delivered at all, canceled, unavailable, or still stuck in transit. A delay doesn't mean anything for those, so I flag them as "Not Delivered" and keep them out of the delay stats, but I don't delete them: an order that never arrived is its own kind of failure and worth counting separately.

(There are also 8 rows marked "delivered" that have no delivery date, a data entry gap. The classifier routes anything

without a date to "Not Delivered", so they can't corrupt the numbers.)

```
master["Days_Difference"] = (
    master["order_estimated_delivery_date"] - master["order_delivered_customer_date"]
).dt.days

missing = master["order_delivered_customer_date"].isna()
print(f"Orders without a delivery date: {missing.sum()}")
print(master.loc[missing, "order_status"].value_counts())
```

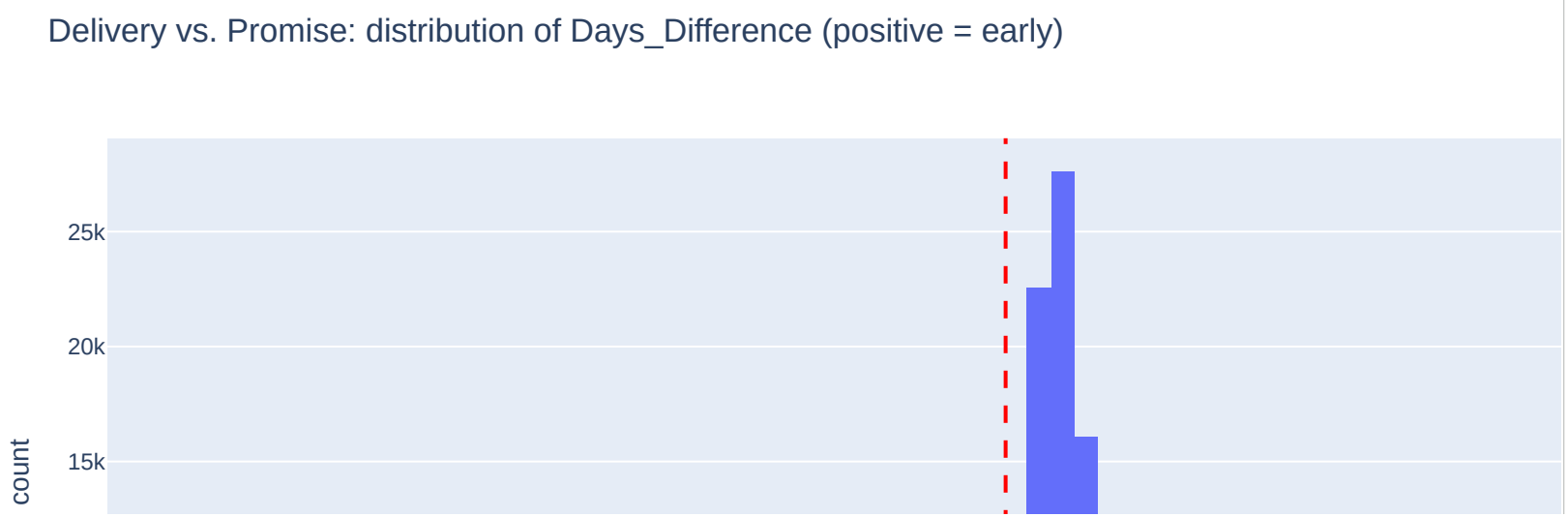
```
Orders without a delivery date: 2965
order_status
shipped        1107
canceled        619
unavailable     609
invoiced        314
processing      301
delivered         8
created           5
approved          2
Name: count, dtype: int64
```

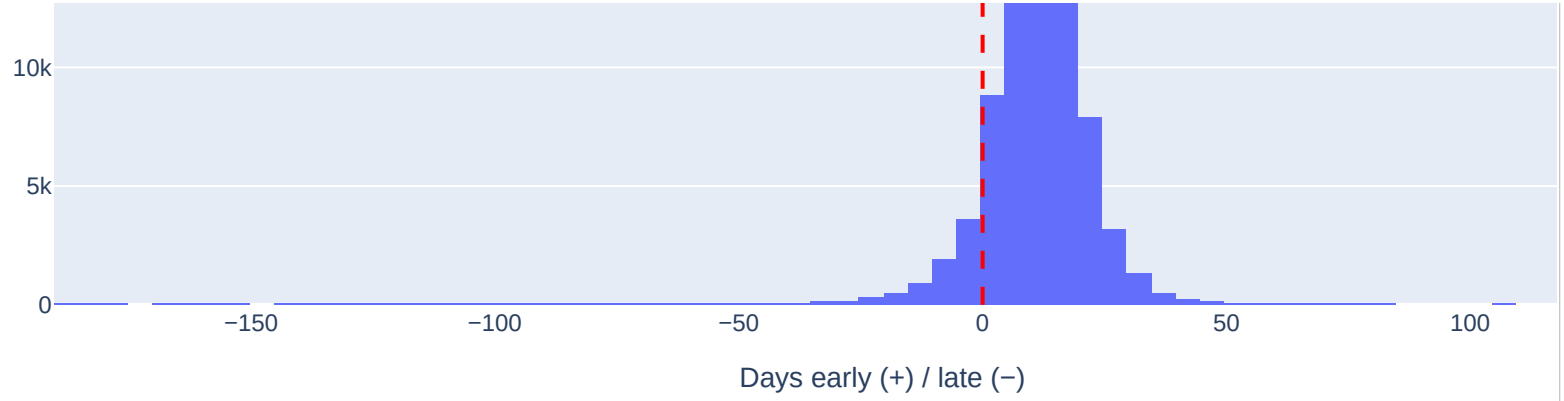
```
def classify(row):
    if pd.isna(row["Days_Difference"]) or row["order_status"] != "delivered":
        return "Not Delivered"
    if row["Days_Difference"] >= 0:
        return "On Time"
    if row["Days_Difference"] >= -5:
        return "Late"
    return "Super Late"

master["delivery_status"] = master.apply(classify, axis=1)

counts = master["delivery_status"].value_counts()
pct = (counts / len(master) * 100).round(1)
print(pd.DataFrame({"orders": counts, "% of all": pct}))
```

```
delivered = master[master["delivery_status"] != "Not Delivered"]
late_rate = (delivered["delivery_status"] != "On Time").mean() * 100
print(f"\nLate rate (delivered orders only): {late_rate:.1f}%")
```

```
                 orders  % of all
delivery_status
On Time           88644      89.1
Super Late         4211       4.2
Late               3615       3.6
Not Delivered      2971       3.0

Late rate (delivered orders only): 8.1%
```

```
import plotly.express as px

fig = px.histogram(
    delivered, x="Days_Difference", nbins=80,
    title="Delivery vs. Promise: distribution of Days_Difference (positive = early)",
    labels={"Days_Difference": "Days early (+) / late (−)"},
)
fig.add_vline(x=0, line_dash="dash", line_color="red")
fig.show()
```





Story 3: Where Are We Failing?

This is the CEO's actual question. I calculate the % of late deliveries per state, using delivered orders only.

One thing to keep in mind while reading the results: Olist's sellers are heavily concentrated around São Paulo. And one thing: some remote states have only a handful of orders (Roraima has 41), so their percentages bounce around so I include order counts alongside every rate so nobody takes a small-sample number too seriously.

What I found: the failures cluster in the **Northeast** , Alagoas is late on almost 1 in 4 orders (23.9%), with Maranhão, Piauí, Ceará and Bahia all well above the 8.1% national average. Interestingly, the far-north Amazon states look *fine* by this metric, not because delivery there is fast, but because their estimates are padded so generously the promise is almost always kept. So this metric measures broken promises, not slow shipping and promises break most where the estimates are calibrated too optimistically. Rio de Janeiro is its own problem: a moderate rate (13.5%) on huge volume, which works out to more late orders than the worst Northeastern states combined.

```
state_stats = (
    delivered.groupby("customer_state")
    .agg(
        orders=("order_id", "count"),
        late_orders=("delivery_status", lambda s: (s != "On Time").sum()),
```

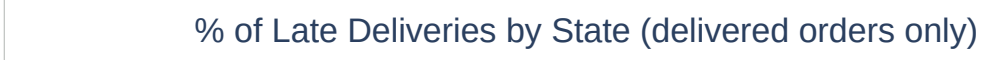
```
        avg_delay_when_late=("Days_Difference", lambda s: -s[s < 0].mean()),
    )
    .assign(late_pct=lambda d: (d["late_orders"] / d["orders"] * 100).round(1))
    .sort_values("late_pct", ascending=False)
)

state_stats
```

customer_state	orders	late_orders	avg_delay_when_late	late_pct
AL	397	95	9.536842	23.9
MA	717	141	10.312057	19.7
PI	476	76	12.592105	16.0
CE	1279	196	14.632653	15.3
SE	335	51	17.196078	15.2
BA	3256	457	11.417943	14.0
RJ	12350	1664	13.149038	13.5
TO	274	35	6.028571	12.8
PA	946	117	12.615385	12.4
RR	41	5	37.400000	12.2
ES	1995	244	10.909836	12.2
MS	701	81	8.000000	11.6
PB	517	57	10.789474	11.0
PE	1593	172	11.645349	10.8
RN	474	51	13.490196	10.8
SC	3546	346	7.991329	9.8

GO	1957	160	10.093750	8.2
DF	2080	147	6.952381	7.1
RS	5344	382	9.712042	7.1
MT	886	60	10.400000	6.8
SP	40494	2387	7.353582	5.9
MG	11354	637	7.844584	5.6
PR	4923	246	7.731707	5.0
AP	67	3	49.333333	4.5
AM	145	6	21.166667	4.1
AC	80	3	19.666667	3.8
RO	243	7	6.571429	2.9

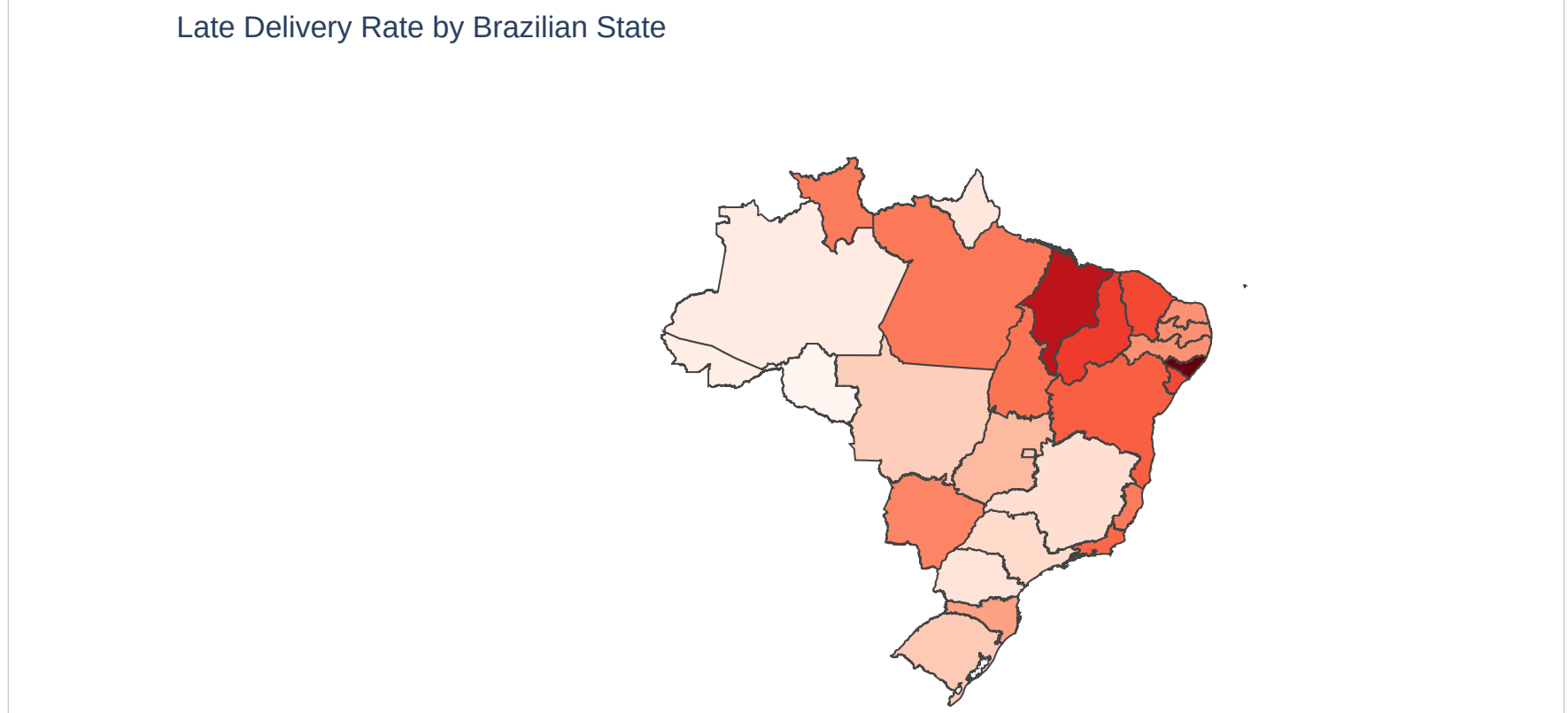
Next steps: Generate code with `state_stats` New interactive sheet

```
fig = px.bar(
    state_stats.reset_index(),
    x="customer_state", y="late_pct",
    hover_data=["orders", "avg_delay_when_late"],
    title="% of Late Deliveries by State (delivered orders only)",
    labels={"late_pct": "% late", "customer_state": "State"},
)
fig.add_hline(y=late_rate, line_dash="dash", line_color="red",
              annotation_text=f"National avg {late_rate:.1f}%")
fig.show()
```




```
import requests

geojson_url = ("https://raw.githubusercontent.com/codeforamerica/click_that_hood/"
               "main/public/data/brazil-states.geojson")
br_states = requests.get(geojson_url).json()

fig = px.choropleth(
    state_stats.reset_index(),
    geojson=br_states,
    locations="customer_state",
    featureidkey="properties.sigla",
    color="late_pct",
    color_continuous_scale="Reds",
    hover_data=["orders"]
```

```
    title="Late Delivery Rate by Brazilian State",
    labels={"late_pct": "% late"},
)
fig.update_geos(fitbounds="locations", visible=False)
fig.show()
```



Story 4: The Sentiment Correlation

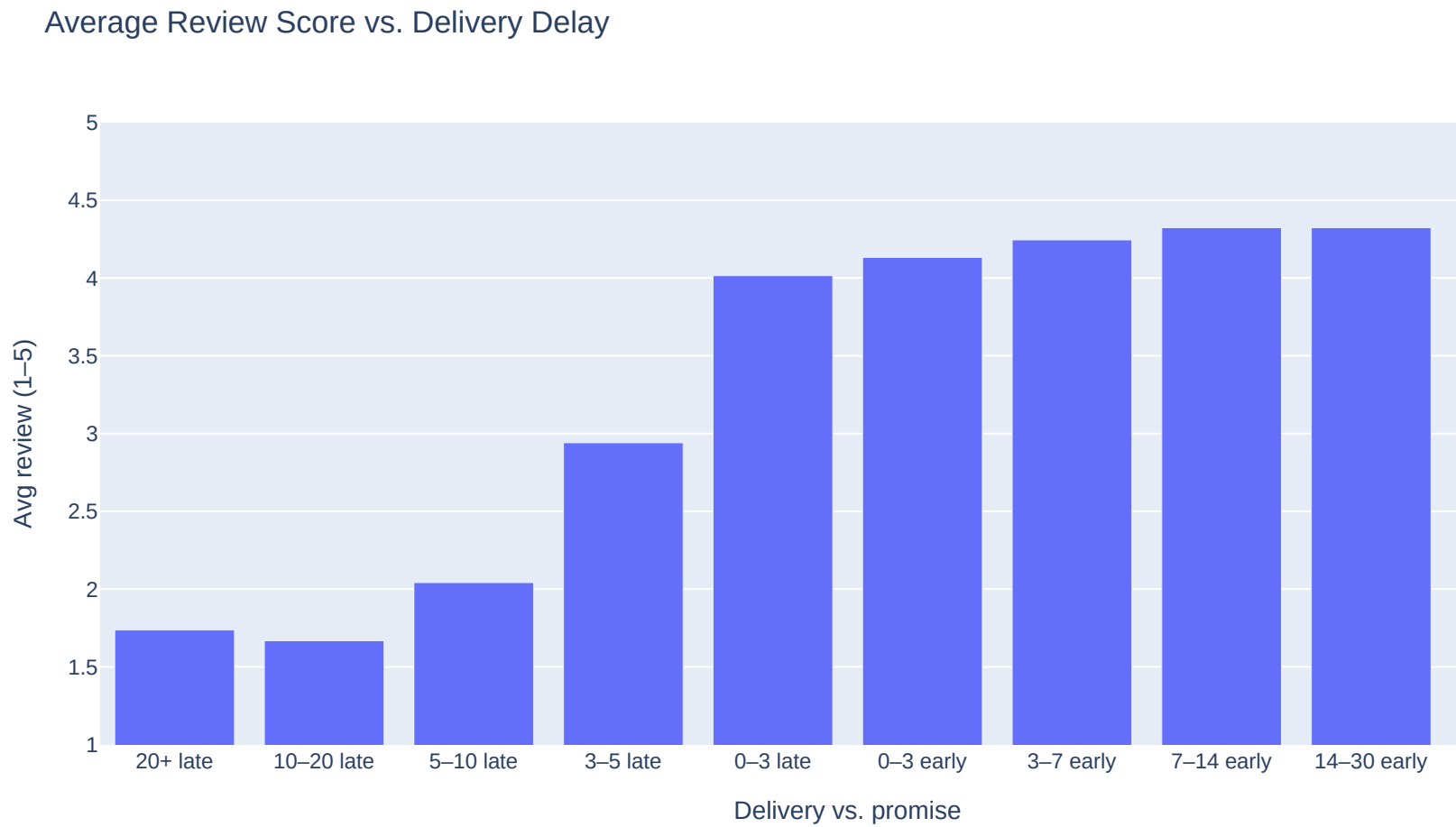
Does lateness actually cause bad reviews? We compare average review score across delivery statuses, then across the full range of delay days, using delivered orders that have a review.

```
reviewed = delivered.dropna(subset=["review_score"])

status_scores = (
    reviewed.groupby("delivery_status")
    .agg(orders=("order_id", "count"), avg_review=("review_score", "mean"))
    .round(2)
    .reindex(["On Time", "Late", "Super Late"])
)
print(status_scores)
```

```
                 orders  avg_review
delivery_status
On Time           88163        4.29
Late               3568        3.46
Super Late         4093        1.78
```

```
bins = [-200, -20, -10, -5, -3, 0, 3, 7, 14, 30, 200]
labels_ = ["20+ late", "10–20 late", "5–10 late", "3–5 late", "0–3 late",
           "0–3 early", "3–7 early", "7–14 early", "14–30 early", "30+ early"]

reviewed = reviewed.assign(
    delay_bucket=pd.cut(reviewed["Days_Difference"], bins=bins, labels=labels_)
)

bucket_scores = (
    reviewed.groupby("delay_bucket", observed=True)
    .agg(avg_review=("review_score", "mean"), orders=("order_id", "count"))
    .reset_index()
)

fig = px.bar(
    bucket_scores, x="delay_bucket", y="avg_review", hover_data=["orders"],
    title="Average Review Score vs. Delivery Delay",
```

```
    labels={"delay_bucket": "Delivery vs. promise", "avg_review": "Avg review (1–5)"},
    range_y=[1, 5],
)
fig.show()
```



The "Translation" Challenge

Product categories are in Portuguese (e.g., `cama_mesa_banho`). We merge the dataset's official `product_category_name_translation.csv` file to get English names (`bed_bath_table`) for the final dashboard.

An order can contain multiple items, so we link **orders → items → products** and keep one category per order (the first item's). This preserves the one-row-per-order guarantee established in Story 1.

```
translation = pd.read_csv(os.path.join(path, "product_category_name_translation.csv"))
items = pd.read_csv(os.path.join(path, "olist_order_items_dataset.csv"))

products_en = products.merge(translation, on="product_category_name", how="left")

order_cats = (
    items.merge(products_en[["product_id", "product_category_name_english"]],
                on="product_id", how="left")
    .drop_duplicates(subset="order_id", keep="first")
    [["order_id", "product_category_name_english"]]
)

master = master.merge(order_cats, on="order_id", how="left")

assert master["order_id"].is_unique, "Translation merge duplicated rows!"
print(f"Master dataset: {master.shape}")
print(f"Orders without a category: {master['product_category_name_english'].isna().sum()}")
print("\nTop 10 categories:")
print(master["product_category_name_english"].value_counts().head(10))
```

```
Master dataset: (99441, 15)
Orders without a category: 2212

Top 10 categories:
product_category_name_english
bed_bath_table           9311
health_beauty            8796
sports_leisure           7681
computers_accessories    6660
furniture_decor          6355
housewares               5829
```

```
housewares                3029
watches_gifts             5601
telephony                 4182
auto                      3880
toys                      3861
Name: count, dtype: int64
```

Story 6 (Candidate's Choice): The Blame Split — Seller vs. Carrier

Knowing *where* we're late isn't enough to fix it. Veridi needs to know *who* is responsible. Every delivery has two legs:

1. **Seller leg:** purchase → package handed to carrier
2. **Carrier leg:** carrier pickup → customer's door

The items table provides `shipping_limit_date` — the deadline for the seller to dispatch. For each **late** order we assign blame:

- **Seller**: handed to carrier after the shipping limit
- **Carrier**: seller dispatched on time, but delivery still missed the promise

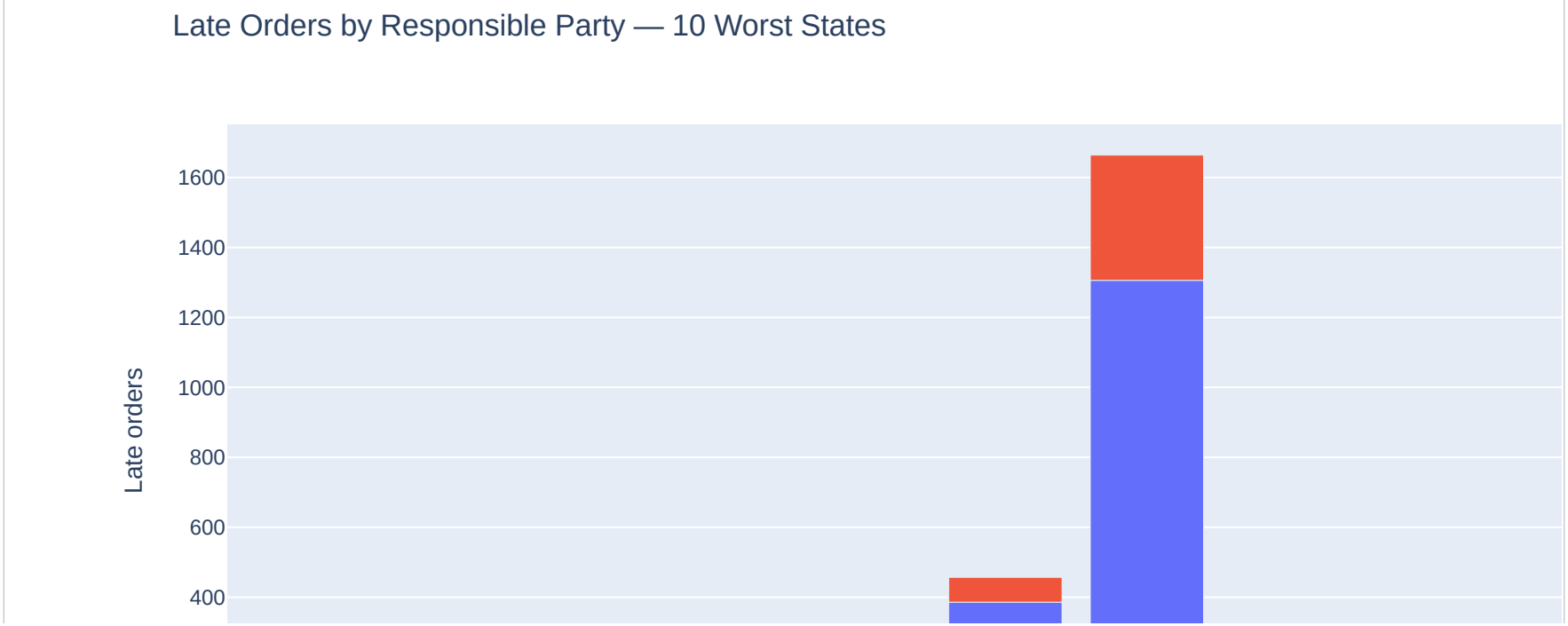
Why this matters to the business: the two verdicts trigger different actions. Seller-caused lateness calls for seller SLAs and penalties; carrier- caused lateness calls for renegotiating carrier contracts or switching carriers on specific routes. Without this split, Veridi risks spending money fixing the wrong side of the chain.

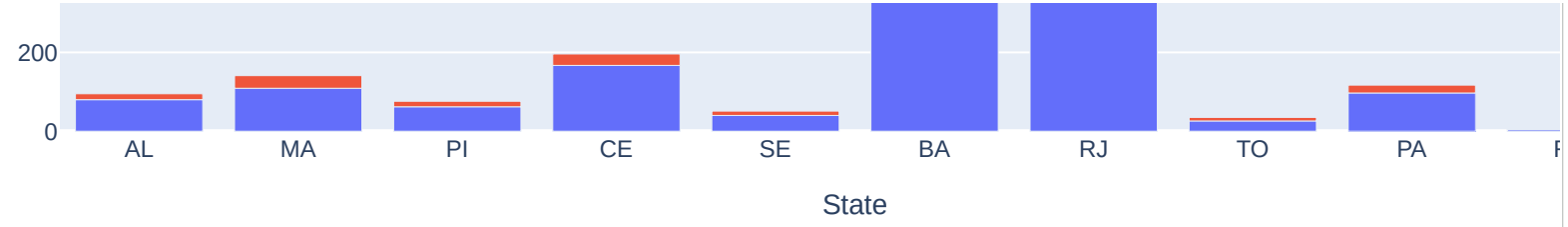
```
# Shipping deadline per order (first item's, consistent with our category logic)
items["shipping_limit_date"] = pd.to_datetime(items["shipping_limit_date"])
order_limits = (
    items.sort_values("shipping_limit_date")
         .drop_duplicates(subset="order_id", keep="first")
         [["order_id", "shipping_limit_date"]]
)

master = master.merge(order_limits, on="order_id", how="left")
assert master["order_id"].is_unique

# The two legs (in days)
```

```
# The two legs (in days)
master["seller_days"]  = (master["order_delivered_carrier_date"]
                          - master["order_purchase_timestamp"]).dt.days
master["carrier_days"] = (master["order_delivered_customer_date"]
                          - master["order_delivered_carrier_date"]).dt.days

def assign_blame(row):
    if row["delivery_status"] not in ("Late", "Super Late"):
        return None
    if pd.isna(row["order_delivered_carrier_date"]) or pd.isna(row["shipping_limit_date"]):
        return "Unknown"
    if row["order_delivered_carrier_date"] > row["shipping_limit_date"]:
        return "Seller"
    return "Carrier"

master["late_blame"] = master.apply(assign_blame, axis=1)

late_orders = master[master["late_blame"].notna()]
blame_counts = late_orders["late_blame"].value_counts()
print(blame_counts)
print(f"\nCarrier share of late orders: "
      f"{(blame_counts.get('Carrier', 0) / len(late_orders) * 100):.1f}%")

# Average leg length, late vs on-time — where does the extra time go?
print(master.groupby("delivery_status")[["seller_days", "carrier_days"]].mean().round(1))
```

```
late_blame
Carrier    5697
Seller     2128
Unknown       1
Name: count, dtype: int64

Carrier share of late orders: 72.8%
                 seller_days  carrier_days
delivery_status
Late                     4.6          16.6
Not Delivered            3.3          13.0
On Time                  2.5           7.4
Super Late               6.0          32.6
```

```
worst_states = state_stats.head(10).index

blame_by_state = (
    late_orders[late_orders["customer_state"].isin(worst_states)]
    .groupby(["customer_state", "late_blame"]).size()
    .reset_index(name="orders")
)

fig = px.bar(
    blame_by_state, x="customer_state", y="orders", color="late_blame",
    category_orders={"customer_state": list(worst_states)},
    title="Late Orders by Responsible Party — 10 Worst States",
    labels={"orders": "Late orders", "customer_state": "State",
            "late_blame": "Responsible"},
)
fig.show()
```






```
from google.colab import files
!jupyter nbconvert --to html "/content/drive/MyDrive/Colab Notebooks/veridi_delivery_audit.ipynb" 2>/d
  || echo "adjust path — see note"
```

```
This application is used to convert notebook files (*.ipynb)
        to various other formats.

        WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options
=======
The options below are convenience aliases to configurable class-options,
as listed in the "Equivalent to" description-line of the aliases.
To see all configurable class-options for some <cmd>, use:
    <cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]
--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]
--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]
--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]
-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]
```

```
--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]
--allow-errors
    Continue notebook execution even if one of the cells throws an error and include the error message
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]
--stdin
    read a single notebook file from stdin. Write the resulting notebook with default basename 'notebo
    Equivalent to: [--NbConvertApp.from_stdin=True]
--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]
--inplace
    Run nbconvert in place, overwriting the existing notebook (only
            relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --Fil
--clear-output
    Clear output of current file and save in place,
            overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --Fil
--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook --Fil
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.exclude_output_pro
--no-input
    Exclude input cells and output prompts from converted document.
            This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporter.exclude_input=Tru
--allow-chromium-download
```